

# EE211: Robotic Perception and Intelligence

## Lecture 1 Introduction

Jiankun WANG

Department of Electronic and Electrical Engineering  
Southern University of Science and Technology

Undergraduate Course, Sep 2026



# Outline

- 1 Course Introduction
- 2 Linux Introduction
- 3 ROS Introduction



# Outline

- 1 Course Introduction
- 2 Linux Introduction
- 3 ROS Introduction



- **Instructor** Jiankun WANG

- Email: wangjk@sustech.edu.cn
- Address: Room 709, South Tower, College of Engineering

- **Teaching Assistants**

- Siyuan Wang, 12012324@mail.sustech.edu.cn
- Zihan Jiang
- Zhaoqing Zeng

- **Lectures**

- Address: Room 224, Teaching Building 1
- Time: Tue 1-2 weekly (1-16)

- **Lab**

- Address: Room 433, South Tower, College of Engineering
- Time: Tue 3-4 weekly (1-16)



- Introduce the commonly used sensors and their working principles in robots, including inertial sensing, GPS and odometry, 3D vision for navigation and grasping tasks, visual servoing, and multi-sensor data fusion.
- Introduce the intelligent planning methods in different robot tasks.
- (TBD) Introduce the commonly used robot learning algorithms.



- Understand the working principle of common sensors.
- Understand basic robot motion and path planning algorithms.
- (TBD) Understand basic robot learning algorithms.
- Use robotic perception and intelligence to complete a specific robot task through teamwork.



- Lectures & Lab & Assignments
- Textbook and Supplementary Readings
  - Siciliano, B., & Khatib, O. (2016). Springer handbook of robotics
  - Lynch, K. M., & Park, F. C. (2017). Modern robotics
  - Thrun, S., Burgard, W., & Fox, D. (2005). Probabilistic robotics
  - Bishop, C. M., & Nasrabadi, N. M. (2006). Pattern recognition and machine learning
- Academic Papers from ICRA, IROS, RAL, TRO, TASE, IJRR



# Course Contents

| No.   | Dates         | Contents                               |
|-------|---------------|----------------------------------------|
| 8     | Sep.9         | Introduction                           |
| 2-4   | Sep.15-Sep.29 | Trajectory Generation, Motion Planning |
| 6-7   | Oct.13-Oct.20 | Basic & advanced planning algorithms   |
| 8-10  | Oct.27-Nov.10 | Different sensors for perception       |
| 11-14 | Nov.17-Dec.8  | Sensor information processing          |
| 15-16 | Dec.15-Dec.22 | Project Q & A and Exam                 |





- Assignment and Sign-in 20%
  - Sign-in or quiz 5%
  - 3 assignments 15%
- Project 30%
  - Conduct real-world robot experiments
- Final Examination 50%
  - Closed-book exam



# Project Description

- Specific task: Conduct real-world robot experiments involving mobile navigation and grasping
- Teamwork: 4? persons in each group
- Evaluation metric: Announced later



## 机器人感知与智能课程 (EE211) 课程成果展

王建坤

机器人智能与感知重点实验室 (rπ Lab)



## 机器人感知与智能课程 (EE211) 课程成果展

王建坤

机器人智能与感知重点实验室 (rπ Lab)



## 机器人感知与智能课程 (EE211) 课程成果展

王建坤

机器人智能与感知重点实验室 (rπ Lab)



# Outline

- 1 Course Introduction
- 2 Linux Introduction
- 3 ROS Introduction



# Operating systems

- **operating system:** Manages activities and resources of a computer.
  - software that acts as an interface between hardware and user
  - provides a layer of abstraction for application developers
- features provided by an operating system:
  - ability to execute programs (and multi-tasking)
  - memory management (and virtual memory)
  - file systems, disk and network access
  - an interface to communicate with hardware
  - a user interface (often graphical)
- **kernel:** The lowest-level core of an operating system.



- The UNIX operating system was born in the late 1960s. It originally began as a one man project led by Ken Thompson of Bell Labs, and has since grown to become the most widely used operating system.
- In the time since UNIX was first developed, it has gone through many different generations and even mutations.
  - Some differ substantially from the original version, like Berkeley Software Distribution (BSD) or **Linux**.
  - Others, still contain major portions that are based on the original source code.
- An interesting and rather up-to-date timeline of these variations of UNIX can be found at <http://www.levenez.com/unix/history.html>.





- Linux: A kernel for a Unix-like operating system.
  - commonly seen/used today in servers, mobile/embedded devices, ...
- GNU(a recursive acronym for "GNU's Not Unix!"): A "free software" implementation of many Unix-like tools
  - many GNU tools are distributed with the Linux kernel
- distribution: A pre-packaged set of Linux software.
  - examples: Ubuntu, Fedora
- key features of Linux:
  - open source software: source can be downloaded
  - free to use
  - constantly being improved/updated by the community



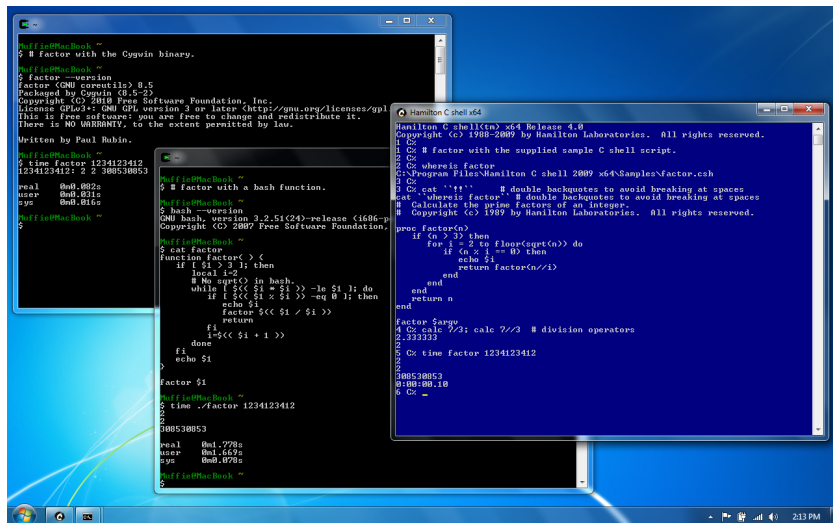
# Linux Distributions



- Shell: An interactive program that uses user input to manage the execution of other programs.
  - A command processor, typically runs in a text window.
  - User types commands, the shell runs the commands
  - Several different shell programs exist. bash-the default shell program on most Linux/Unix systems. Other shells: Bourne, csh, tsch
- Why should I learn to use a shell when GUIs exist?



# Shell Example



```
Puffie@MacBook ~
$ # Factor with the Cygwin binary.

Puffie@MacBook ~
$ factor --version
factor (GNU coreutils) 8.5
Packaged by Cygwin (8.5-22)
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later (http://gnu.org/licenses/gpl)
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by Paul Rubin.

Puffie@MacBook ~
$ time factor 1234123412
1234123412: 2 2 308530853

real    0m0.082s
user    0m0.031s
sys     0m0.016s

Puffie@MacBook ~
$ # factor with a bash function.

Puffie@MacBook ~
$ hash --version
GNU hash, version 3.2.51(24)-release (1686-p
Copyright (C) 2007 Free Software Foundation,

Puffie@MacBook ~
$ cat factor
function factor() {
    local i=2
    # No sqrt() in hash.
    while [ $(( $i * $i )) -le $1 ]; do
        if [ $(( $1 % $i )) -eq 0 ]; then
            echo $i
            factor $(( $1 / $i ))
            return
        fi
        i=$(( $i + 1 ))
    done
    echo $1
}

factor $1

Puffie@MacBook ~
$ time ./factor 1234123412
2.22
308530853

real    0m1.278s
user    0m1.569s
sys     0m0.078s

Puffie@MacBook ~

Hamilton C shell(x64)
Hamilton C shell(x64) Release 4.0
Copyright (c) 1988-2009 by Hamilton Laboratories. All rights reserved.
1 C%
1 C% # factor with the supplied sample C shell script.
2 C%
2 C% whereis factor
3 C% cat '```' # double backquotes to avoid breaking at spaces
3 C% cat 'whereis factor' # double backquotes to avoid breaking at spaces
# Calculate the prime factors of an integer.
# Copyright (c) 1989 by Hamilton Laboratories. All rights reserved.
4 C%
4 C% proc factor(n)
5 C%     if (n > 3) then
6 C%         for i = 2 to floor(sqrt(n)) do
7 C%             if (n % i == 0) then
8 C%                 echo $i
9 C%                 return factor(n//i)
10 C%             end
11 C%         end
12 C%     end
13 C%     return n
14 C% end
15 C%
16 C% Factor $argv
17 C% 4 C% calc 7/3; calc 7//3 # division operators
18 C% 2.333333
19 C%
20 C%
21 C% C% time factor 1234123412
22 C%
23 C% 308530853
24 C% 0:00:00.10
25 C% =
26 C%
```

# Graphical User Interfaces (GUIs)

- When you logon locally, you are presented with graphical environment.
- You start at a graphical login screen. You must enter your username and password. You also have the option to choose from a couple session types. Mainly you have the choice between Gnome and KDE.
- Once you enter in your username and password, you are then presented with a graphical environment that looks like one of the following...



- You also have access to some UNIX servers as well.
  - You can logon from virtually any computer that has internet access whether it be Windows, Mac, or UNIX itself.
- In this case you are communicating through a local terminal to one of these remote servers.
  - All of the commands actually execute on the remote server.
  - It is also possible to open up graphical applications through this window, but that requires a good bit more setup and software.



# Why use a shell?

- faster
- work remotely
- programmable
- customizable
- repeatable



# Shell commands

| command | description                                                 |
|---------|-------------------------------------------------------------|
| exit    | logs out of the shell                                       |
| ls      | lists files in a directory                                  |
| pwd     | <u>p</u> rint the current <u>w</u> orking <u>d</u> irectory |
| cd      | <u>c</u> hanges the working <u>d</u> irectory               |
| man     | brings up the manual for a command                          |

```
$ pwd
/homes/iws/rea
$ cd CSE391
$ ls
file1.txt file2.txt
$ ls -l
-rw-r--r-- 1 rea    fac_cs 0 2016-03-29 17:45 file1.txt
-rw-r--r-- 1 rea    fac_cs 0 2016-03-29 17:45 file2.txt
$ cd ..
$ man ls
$ exit
```





# Relative directories

| directory         | description                                                                      |
|-------------------|----------------------------------------------------------------------------------|
| .                 | the directory you are in ("working directory")                                   |
| ..                | the parent of the working directory<br>(../.. is grandparent, etc.)              |
| ~                 | your <u>home</u> directory<br>(on many systems, this is /home/ <i>username</i> ) |
| ~ <i>username</i> | <i>username</i> 's <u>home</u> directory                                         |
| ~/Desktop         | your desktop                                                                     |



# Directory commands

| command | description                                                 |
|---------|-------------------------------------------------------------|
| ls      | list files in a directory                                   |
| pwd     | <u>p</u> rint the current <u>w</u> orking <u>d</u> irectory |
| cd      | <u>c</u> hanges the working <u>d</u> irectory               |
| mkdir   | create a new directory                                      |
| rmdir   | delete a directory (must be empty)                          |

- some commands (cd, exit) are part of the shell ("builtins")
- others (ls, mkdir) are separate programs the shell runs



- OS does not have to use a graphical interface.
  - The OS itself (the kernel) is incredibly small.
  - The GUI just another application (or set of applications) that can be installed and run on top the existing text-based OS.
- File system differences.
  - Windows typically uses FAT32 or NTFS file systems; Linux typically uses the ext2 or ext3 file systems.
  - Windows lists all drives separately (A:,C:,D:, etc. . . ), with “My Computer” at the highest level; UNIX starts its highest level at “/” and drives can be mounted anywhere underneath it.



# Outline

- 1 Course Introduction
- 2 Linux Introduction
- 3 ROS Introduction**



- Lack of standards
- Little code reusability
- Keeping reinventing (or rewriting) device drivers, access to robot's interfaces, management of onboard processes, inter-process communication protocols, ...
- Keeping re-coding standard algorithms
- New robot in the lab (or in the factory) → start re-coding (mostly) from scratch

# Robot Operating System (ROS)

- ROS is an open-source robot operating system
- A set of software libraries and tools that help you build robot applications that work across a wide variety of robotic platforms
- Originally developed in 2007 at the Stanford Artificial Intelligence Laboratory and development continued at Willow Garage
- Since 2013 managed by OSRF (Open Source Robotics Foundation)



- **The operating system side**, which provides standard operating system services such as: hardware abstraction
  - low-level device control
  - implementation of commonly used functionality
  - message-passing between processes
  - package management
- **A suite of user contributed packages** that implement common robot functionality such as SLAM, planning, perception, vision, manipulation, etc.



- **Peer to Peer:** ROS systems consist of many small programs (nodes) which connect to each other and continuously exchange messages
- **Tools-based:** There are many small, generic programs that perform tasks such as visualization, logging, plotting data streams, etc.
- **Multi-Lingual:** ROS software modules can be written in any language for which a client library has been written. Currently client libraries exist for C++, Python, LISP, Java, JavaScript, MATLAB, Ruby...
- **Thin:** The ROS conventions encourage contributors to create stand-alone libraries/packages and then wrap those libraries so they send and receive messages to/from other ROS modules.
- **Free & open source, community-based, repositories**









# TurtleBot 4 - An Out of This World Demonstration

